

```

graph Exp["Exploratory"]
    E1["Chaos Testing"]
    E2["What-if Scenarios"]
end

subgraph P3["Pillar 3: Observability"]
    O1["Test Environment Monitoring"]
    O2["Production Monitoring"]
    O3["Performance Prediction"]
end

subgraph P4["Pillar 4: Data Store"]
    DS1["Realistic Customer Model"]
    DS2["Data Generation Tools"]
    DS3["Performance Baselines"]
end

Portal --> P1
Portal --> P2
Portal --> P3
Portal --> P4

P4 --> P2
P3 --> P2
P3 --> P1

```

## Approach

Deploy verification checks can be divided into three approaches:

1. Improving GitLab's reporting to alert when a problem is detected (is the instance able to connect to S3)
2. Adding specific checks that can be run in a pipeline to detect if a component is configured correctly (is S3 configured correctly to be connected to)
3. Once those two are done, we can make the determination if it is worth running an E2E test to validate that the GitLab instance is using the component as intended (is GitLab able to push a run result to S3 for future analysis) NOTE: in virtually all use cases, this path will be redundant with GitLab functionality tests that are run earlier in the development cycle and should only be run if we have indentified that there's a specific need to test GitLab functionality as part of the deploy instead of configuration.

## Checks

This is a list of the checks that we have indentified as being ones that we are working on developing tests to support:

| Check Name | Description | Notes |
|------------|-------------|-------|
| -----      | -----       | ----- |

## E2E tests

Most of the testing this page documents will be non E2E tests, this section provides a space for us to notate scenarios we have uncovered that are best covered by an E2E test.

| Scenario | Description | Test that covers |
|----------|-------------|------------------|
| -----    | -----       | -----            |

---

## SECTION: engineering/testing/performance-tools.md

### Overview

Performance Testing is a broad discipline that includes various approaches to evaluate a system's performance characteristics. Load Testing, while often considered synonymous with Performance Testing is one of many approaches to Performance Testing. There are other approaches that enable testing performance throughout the development lifecycle, from early development to production monitoring.

Use the decision tree below to find the right performance testing approach for your needs.

mermaid

flowchart LR

```
subgraph PTK["Performance Testing Kit"]
    GPT[["GitLab Performance Tool"]]
    CPT[["Component Performance Testing (alpha)"]]
    GBPT[["GitLab Browser Performance Tool"]]
end
```

START((Start))

ENV{Testing API/backend\nperformance?}

UI{Testing page load\nperformance?}

COMPONENT{Testing a specific\ncomponent in isolation?}

START --> ENV

ENV -- yes --> GPT

ENV -- no --> UI

UI -- yes --> GBPT

UI -- no --> COMPONENT

COMPONENT -- yes --> CPT

COMPONENT -- no --> GPT

%% Class definition

classDef decision fill:f5f7f6,stroke:333,stroke-width:1px,rx:5px;

classDef tool fill:F28C6B,stroke:333,stroke-width:1px,color:white,rx:5px;

```

classDef start fill:03822d,stroke:333,stroke-width:1px,color:white,rx:10px;
classDef kitStyle fill:f0f8ff,stroke:4a90e2,stroke-width:2px,color:333;

class UI,ENV,COMPONENT decision;
class GBPT,CPT,GPT tool;
class START start;
class PTK kitStyle;

%% Tool links
click GPT "https://gitlab.com/gitlab-org/quality/performance/-/blob/main/docs/quickstart.md"
click GBPT "https://gitlab.com/gitlab-org/quality/performance-sitespeedrunning-tests"
click CPT "https://gitlab.com/gitlab-org/quality/component-performance-testing"

%% Decision node links
click UI "Testing-Page-Load-Performance"
click ENV "Testing-API-or-Backend-Performance"
click COMPONENT "Testing-Components-in-Isolation"

```

## Testing API or Backend Performance

Use GitLab Performance Tool when you want to test how your APIs, database queries, or backend services perform under load. This includes testing response times, throughput, and system behavior under various load conditions.

When to use:

- Testing REST API endpoints
- Validating database query performance
- Load testing backend services
- Reference architecture validation

## Testing Components in Isolation

Use Component Performance Testing Tool to run automated load tests on individual services or components at the Merge Request level, providing early feedback on performance changes before they reach production. It does this by running tests against internal API endpoints rather than end-user API endpoints.

When to use:

- Testing containerized services that can be deployed independently (like Gitaly, AI Gateway, Registry)
- Detecting performance regressions in MRs before merge
- Validating that component changes don't introduce throughput bottlenecks
- Getting fast feedback on API response times and resource utilization
- Testing component-specific caching strategies and configuration changes

What it tests:

- Component throughput and response times
- Resource utilization (CPU, memory, network I/O)
- Error handling performance
- Configuration-related performance impacts

What it doesn't test:

- Integration bottlenecks between components
- Production-scale data volume issues
- End-to-end system performance

> .. Alpha Status: Component Performance Testing is currently in alpha trials. If you want to use it, please reach out in Slack to gperformanceenablement.

Prerequisites for adoption:

- Component must be containerized and deployable in isolation
- Must expose testable interfaces (HTTP APIs, gRPC, etc.)
- Should support testing with mocked dependencies

### Testing Page Load Performance

Use GitLab Browser Performance Tool to measure how fast your pages load for users, including metrics like Time to First Byte, Largest Contentful Paint, and other Core Web Vitals.

When to use:

- Testing frontend performance
- Measuring page load times
- Validating user experience metrics
- Browser-based performance testing

### Future improvements

These tools have not been implemented and will complement the Performance Testing Kit and enhance our ability to detect performance problems early.

mermaid

flowchart LR

START((Start))

UNIT[[Performance checks in Unit Tests]]

PROFILE[[Profiling tools]]

OBSERVETEST[[Observability based Performance Testing]]

SPECS{Testing with\nnew unit tests?}

BUILT{Testing during\ndevelopment?}

OBSERVABILITY{Analyzing live\nperformance data?}

START --> BUILT

BUILT -- no --> OBSERVABILITY

BUILT -- yes --> SPECS  
SPECS -- yes --> UNIT  
SPECS -- no --> PROFILE

OBSERVABILITY -- yes --> OBSERVETEST

%% Class definition

classDef decision fill:f5f7f6,stroke:333,stroke-width:1px,rx:5px;

classDef tool fill:F28C6B,stroke:333,stroke-width:1px,color:white,rx:5px;

classDef start fill:03822d,stroke:333,stroke-width:1px,color:white,rx:10px;

class BUILT,SPECS,OBSERVABILITY decision;

class PROFILE,UNIT,OBSERVETEST tool;

class START start;

%% Tool tooltips with links

click PROFILE "profiling-tools"

click OBSERVETEST

"<https://handbook.gitlab.com/handbook/engineering/testing/observabilityperformance/>"

click UNIT "performance-unit-testing" "Add performance assertions and benchmarks directly within your unit test suite for fast feedback"

%% Decision node tooltips

click OBSERVABILITY "Analyzing-live-performance-data"

click BUILT "Testing-during-development"

click SPECS "Testing-with-new-unit-tests"

## Testing with new unit tests

Add performance assertions directly to your unit tests to catch performance regressions early in development. This provides fast feedback on code changes without requiring separate performance test suites.

When to use:

- Writing new unit tests and want to include performance validation
- Adding performance checks to existing test coverage
- Ensuring critical methods maintain acceptable performance thresholds
- Catching performance regressions during code review

Example approaches:

- Execution time assertions (method completes under X milliseconds)
- Memory allocation limits (method allocates fewer than Y objects)
- Database query count validation

## Testing during development

Use lightweight profiling tools while actively developing to understand performance characteristics of your code before it reaches production.

When to use:

- Optimizing algorithm performance during development
- Understanding memory usage patterns in new features
- Identifying performance bottlenecks in work-in-progress code
- Getting quick feedback on code changes without full test suites

Example tools:

- Code profilers for CPU and memory analysis
- Database query analyzers
- Benchmarking utilities for comparing implementations

Analyzing live performance data

Leverage existing monitoring and observability data to identify performance issues and validate improvements using real production metrics.

When to use:

- Investigating performance issues reported by users
- Validating that performance improvements are effective in production
- Understanding real-world performance patterns
- Correlating code changes with production performance metrics

Example approaches:

- Dashboard analysis of key performance indicators
- Log-based performance trend analysis
- Correlation of deployment events with performance changes

Performance Unit Testing

Performance unit testing allows developers to evaluate and enforce the performance characteristics of their code at the unit level. This approach provides fast feedback on performance during development, helping catch performance regressions early in the development lifecycle.

Using rspec-benchmark

We have rspec-benchmark included in our Gemfile. It is a gem that provides RSpec matchers for performance testing. It offers various matchers to assert on different performance aspects such as execution time, iterations per second, allocation counts, and memory usage.

<details>

<summary>Example Test Case</summary>

Here's a complete example of using rspec-benchmark to test the performance of a method:

```
ruby
require 'spechelper'

RSpec.describe UserFinder do
  describe 'findactive' do
    it 'performs query under 50ms' do
      users = createlist(:user, 100, status: :active)

      expect {
        UserFinder.new.findactive
      }.to performunder(50).ms
    end

    it 'allocates less than 20 objects' do
      users = createlist(:user, 100, status: :active)

      expect {
        UserFinder.new.findactive
      }.to performallocation(count: 1..20)
    end

    it 'scales linearly with number of users' do
      expect do |n, i|
        users = createlist(:user, n, status: :active)
        UserFinder.new.findactive
      end.to performlinear.inrange(10..100).sample(5)
    end
  end
end
```

</details>

## Profiling Tools

We already use profiling tools (i.e. rubocop) in our pipelines to ensure that we meet coding guidelines and avoid common problematic patterns. Several performance focused ones that are in our codebase:

1. ruby-prof: A comprehensive profiling solution that supports both flat and graph profiles. ruby-prof can measure CPU time, memory allocation, and object creation.
2. stackprof: A sampling call-stack profiler. It's designed to be a faster and more memory-efficient alternative to ruby-prof for certain use cases.
3. memoryprofiler: A memory profiler that provides detailed information about memory usage, including object allocation and retention. documentation in our performance guidelines.
4. rbspy: A sampling profiler for Ruby, documentation in our sidekiq troubleshooting docs
5. derailedbenchmarks: A set of benchmarks that measure various aspects of Rails application performance, including memory usage and load time. documentation in our performance guidelines.

6. benchmark-ips: benchmarks a blocks iterations/second
7. rspecprofiling: collects data on spec execution times, documentation from our performance guidelines.

Some approaches to using these tools are detailed on the profiling page

## References

### External References

| Page | Description                                                                                                                              |
|------|------------------------------------------------------------------------------------------------------------------------------------------|
| ---- | -----                                                                                                                                    |
|      | Netflix performance testing   Blog post about performance testing at Netflix                                                             |
|      | Automation Pyramid Model for Performance Testing Process   Blog post looking into the test pyramid for performance testing               |
|      | Continuous Performance Testing: A Comprehensive Guide   Blog post on Continuous Performance Testing                                      |
|      | 3 Challenges to Effective Performance Testing in Continuous Integration   Blog post on challenges implementing performance testing in CI |
|      | When is the Best Time to Start Performance Testing?   Blog post on when to do performance testing                                        |

### Internal References

### Projects

| Project | Description                                                                                           |
|---------|-------------------------------------------------------------------------------------------------------|
| ----    | -----                                                                                                 |
|         | GPT   The GitLab Performance Tool (GPT) provides load testing for GitLab instances                    |
|         | GBPT   SiteSpeed CI pipelines for browser performance testing                                         |
|         | CPT (Alpha)   Component-level performance testing for individual services (currently in alpha trials) |
|         | sitespeed-measurement-setup   Setup to measure performance on GitLab websites through sitespeed.io    |

### Documentation pages

| Page | Description                                                                                                   |
|------|---------------------------------------------------------------------------------------------------------------|
| ---- | -----                                                                                                         |
|      | Performance Strategy & Measurement   GitLab's overall performance strategy, targets, and measurement approach |
|      | Profiling page   Documentation on profiling approaches for GitLab                                             |
|      | Observability for stage groups   Documentation on Observability focused at Stage Groups                       |
|      | Performance Bar   Performance Bar for analyzing performance in running GitLab instances                       |
|      | Dev Performance Guidelines   Developer-focused Performance Guidelines                                         |
|      | Merge Request Performance Guidelines   Performance guidelines specific to Merge Requests                      |
|      | Platform Triage Dashboard   Starting point dashboard for investigating performance issues                     |

---



## ## SECTION: engineering/testing/pipeline-triage.md

### Overview

These guidelines gives GitLab team members on pipeline triage an idea on the priorities and processes that come with this responsibility. This builds from the information provided in On-Call Rotation.

This guide is an extension of the Broken master engineering workflow and is intended to provide a more specific guide on how to triage end-to-end test pipeline failures. As a first step to identifying and resolving a broken master incident, please follow the steps in the broken master process.

The Pipeline triage DRI is responsible for analyzing and debugging test pipeline failures. Please refer to the DRI weekly rotation schedule to know who the current DRIs are.

### NOTE:

For information regarding debugging test pipeline failures, check out Debugging Failing E2E Tests and Test Pipelines

### General guidelines

- Fix tests failing in master before other development work: Failing tests on master are treated as the highest priority relative to other development work, e.g., new features. Note that for pipeline triage DRIs, triage and reporting takes priority over fixing tests.
- Follow the pipeline triage guidelines for investigating, reporting, and resolving test failures
- Flaky tests are quarantined until proven stable: A flaky test is as bad as no tests or in some cases worse due to the effort required to fix or even re-write the test. As soon as it is detected, it is quarantined immediately to stabilize CI, fixed as soon as possible, and monitored until it is fixed.
- Close test failure issue (e.g example issue) when the test is moved out of quarantine: Quarantine issues should not be closed unless tests are moved out of quarantine.
- Quarantine issues should be assigned and scheduled: To ensure that someone is owning the issue, it should be assigned with a milestone set and have appropriate ~"quarantine", quarantine with type (e.g. ~"quarantine::bug") and failure with type (e.g. ~"failure::bug") labels.
- Make relevant stage group aware: When a test fails no matter the reason, an issue with related product group label (e.g. ~"group::ide") should be created and made known to the relevant product stage group as soon as possible.

In addition to notifying that a test in their domain fails, enlist help from the group as necessary.

- Failure due to bug: If one or multiple test failure(s) is a result of a bug, create a bug issue and provide as much details as possible (e.g. using issue's Bug template, provide steps to reproduce, relevant screenshots, etc.). Link the all related test failure issues to the bug issue. Apply ~"type::bug", severity, priority, product group, feature category, etc. labels to ensure a fix is scheduled in a timely manner.

Test failure issues are used for tracking and investigating purposes, they should not have ~"type::bug" label. If the test failure is a result of a bug, apply ~"failure::bug" label instead.

- Everyone can fix a test, the responsibility falls on the last who worked on it: Anyone can fix a failing/flaky test, but to ensure that a quarantined test isn't ignored, the last engineer who worked on the test is responsible for taking it out of quarantine.

## Triage flow

The flow of triaging the pipelines as a decision tree (nodes link to the relevant sections of the handbook)

mermaid

flowchart TB

%% nodes

slackchannels{check slack channel}

nextchannel(move to next slack channel)

failedpipeline(identify failure)

existingissue{issue\nalready\nreported?}

newissue(create issue if does not already exist)

tagpipeline(notate issue in comment on pipeline run)

incident{incident\nworthy?}

notifyincident(Notify about escalation)

openincident(Open incident)

updateincident(Update teams about incident)

eyes(tag pipeline run with :eyes: emoji)

fireengine(replace :eyes: with :fireengine: emoji)

boom(replace :eyes: with :boom: emoji)

anotherfailure{more\nfailures?}

investigate(investigate root cause for found failures)

fixtests(fix tests if possible)

quarantinetests(quarantine tests if necessary)

monitorincident(participate in incident process)

tagissueforreport(add your emoji to issue for the DRI gem)

publishresults(publish your results to the triage issue with the DRI gem)

drihandoff(handoff to next DRI anything that is still in flight)

%% external links

click failedpipeline "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/review-the-failure-logs>"

click newissue "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/create-an-issue>"

click existingissue "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/known-failures>"

click investigate "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/investigate-the-root-cause>"

click incident

"<https://handbook.gitlab.com/handbook/engineering/infrastructure/incident-management/report-an-incident-via-slack>"

click notifygroups "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/notify-group-in-all-cases>"

click fixtests "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/fixing-the-test>"

click quarantinetests "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/quarantining-tests>"  
 click openincident "<https://handbook.gitlab.com/handbook/engineering/infrastructure/incident-management/failure-needs-escalation>"  
 click tagpipeline "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/linking-issue>"  
 click eyes "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/emoji-used>"  
 click fireengine "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/emoji-used>"  
 click boom "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/emoji-used>"  
 click tagissueforreport "<https://gitlab.com/gitlab-org/ruby/gems/dri4-configuration>"  
 click publishresults "<https://gitlab.com/gitlab-org/ruby/gems/dri4-publish>"  
 click notifyincident "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/failure-needs-escalation>"  
 click updateincident "<https://handbook.gitlab.com/handbook/engineering/infrastructure-platforms/developer-experience/debugging-end-to-end-test-failures/notify-group-in-all-cases>"

%% diagram

slackchannels -->|failed pipeline run| eyes  
 slackchannels -->|no failed pipeline runs| nextchannel

openincident --> nextchannel  
 anotherfailure -->|no| nextchannel  
 incident -->|yes| notifyincident

nextchannel --> investigate  
 nextchannel --> publishresults

subgraph report the failure  
 eyes --> failedpipeline  
 failedpipeline --> existingissue  
 existingissue -->|new failure| newissue  
 existingissue -->|existing issue| fireengine  
 newissue --> boom  
 boom --> notifygroups  
 notifygroups --> incident  
 incident -->|no| tagissueforreport  
 tagissueforreport --> tagpipeline

fireengine --> tagissueforreport  
 tagpipeline --> anotherfailure  
 anotherfailure -->|yes| failedpipeline  
 end

subgraph escalate issue  
 notifyincident --> openincident  
 openincident --> updateincident

```
updateincident --> monitorincident  
end
```

```
subgraph follow up on test failures  
  investigate --> fixtests  
  investigate --> quarantinetests  
end
```

```
subgraph end of your day  
  publishresults --> drihandoff  
end
```

## How to triage a QA test pipeline failure

The general triage steps are:

- Report the failure
- Review the failure logs
- Investigate the root cause
- Classify and triage the test failure
- Notify relevant groups about the failure

After triaging failed tests, possible follow up actions are:

- Fixing tests
- Quarantining tests
- Dequarantining tests
- Re-evaluating tests

### Report the failure

Your priority is to make sure we have an issue for each failure, and to communicate the status of its investigation and resolution. When there are multiple failures to report, consider their impact when deciding which to report first. See the pipeline triage responsibilities for further guidance.

If there are multiple failures we recommend that you identify whether each one is new or old (and therefore already has an issue open for it). For each new failure, open an issue that includes only the required information. Once you have opened an issue for each new failure you can investigate each more thoroughly and act on them appropriately, as described in later sections.

The reason for reporting all new failures first is to allow faster discovery by engineers who might find the test failing in their own merge request test pipeline. If there is no open issue about that failure, the engineer will have to spend time trying to figure out if their changes caused it.

Known failures should be linked to the current pipeline triage report. However, issues can be opened by anyone and are not linked automatically, so be sure to confirm there is no existing

issue before creating one.

1. Search for existing issues that have already been created with the failure:: label. By order of likelihood:

- 1. failure::investigating
- 1. failure::test-environment
- 1. failure::broken-test
- 1. failure::flaky-test
- 1. failure::stale-test
- 1. failure::bug
- 1. failure::external-dependency

1. If the issue has already been reported please use the existing issue to track the latest status.

1. If there is no existing issue for the failure, please create an issue using one of classification labels via the steps below.

In the relevant Slack channel:

- 1. Apply the :eyes: emoji to indicate that you're investigating the failure(s).
- 1. If there's a system failure (e.g., Docker or runner failure), retry the job and apply the :retry: emoji. Read below for examples of system failures.
- 1. If an issue exists, add a :fireengine: emoji. It can be helpful to reply to the failure notification with a link to the issue(s), but this isn't always necessary, especially if the failures are the same as in the previous pipeline and there are links there.
- 1. If it is a new failure issue, add a :boom: emoji.

Checkout the list of Slack channels for the pipeline-related channels.

### Create an issue

Please use this step if there are no issues created to capture the failure. If there is already an issue please skip this step.

1. Create an issue for the test or system failure (if retrying the job does not resolve the latter) in <https://gitlab.com/gitlab-org/gitlab/issues> using the QA failure template. For failures in CustomersDot tests, open an issue in CustomersDot project.

- Apply the ~"type::ignore" label to the issue until the investigation is complete and an issue type is determined.
- Inform the counterpart SET about the failure.
- For system failures, it may make sense to open an issue in a different project such as Omnibus GitLab, GitLab QA, or GitLab Runner.
- For staging environment-related failures, you can post a question in infrastructure-lounge, or open an issue in the infrastructure project
- Ask for help in sdeveloperexperience if you're unsure where to file the issue.

1. In the relevant Slack channel, add the :boom: emoji and reply to the failure notification with a link to the issue.

1. Add the issue as a related issue to the current pipeline triage report. If multiple issues are the result of 1 bug, add the bug issue to the report instead.

Review the failure logs

The aim of this step is to understand the failure. The results of the investigation will also let you know what to do about the failure. Update the failure issue with any findings from your review. For more information about the failure logs, check out Debugging Failing Tests and Test Pipelines

### Investigate the root cause

Depending on your level of context for the test and its associated setup, you might feel comfortable investigating the root cause on your own, or you might get help from other SETs right away.

When investigating on your own, we suggest spending at most 20-30 minutes actively trying to find the root cause (this excludes time spent reporting the failure, reviewing the failure logs, or any test setup and pipeline execution time). After that point, or whenever you feel out of ideas, we recommend asking for help to unblock you.

Note: Please avoid logging in via gitlab-qa and all the other bot accounts on Canary/Production. They are monitored by SIRT and will raise an alert if someone uses them to log in. If it is really needed to log in with these accounts, please give a quick heads-up in security-division that someone is logging into the bot and tag @sirt-members for awareness.

Below is the list of the common root causes in descending order of likelihood:

1. Code changes: Check if the new code was deployed to the environment.
  - Find the diff between current and previous GitLab versions using this example <https://gitlab.com/gitlab-org/security/gitlab/-/compare/startcommitsha...endcommitsha> to see if there was a change that could have affected the test.
2. Feature Flag: Check if a new feature flag is enabled in the environment.
  - When a feature flag is enabled, it's being reported to specific QA pipeline Slack channel. This also triggers a Full QA job and it may help to identify which specific feature flag caused the failure.
  - A list of logs containing details on both recent and historic feature flag changes can also be viewed by visiting the feature-flag-log project. Each time a feature flag is updated, a new issue is generated in the project with helpful information such as when the feature flag was changed, who performed the update and on which environment. The project contains several host labels that can help filter by environment when searching through issues (ex: ~host::staging.gitlab.com)
  - You can also reference this dashboard for a visual representation of feature flag statuses.
3. Environment / Infrastructure: If there were no code or feature flag changes and the environment has flaky errors, first start with analyzing Sentry errors and Kibana logs to further investigate the issue.
  - Review the incidents-dotcom channel to check if any ongoing incidents may be contributing to the failures.
  - If validatecanary! check is failing, check if canary is not disabled on the environment by running  
/chatops run canary --production on production or /chatops run canary --staging on staging Slack channels. If canary is enabled, each server should report a few connections UP.  
There is known intermittent issue with the traffic not getting directed to canary even though the gitlabcanary=true cookie is set.

- GitLab's Tamland may also be a helpful resource to review. Tamland is used to help forecast the utilization and saturation of various services, such as Sidekiq. For example, if high saturation is predicted, this could surface as flaky behavior in our tests due to performance degradation from that service. More information about Tamland can be found [here](#).

- You may also reach out to the Infrastructure team at [infrastructure-lounge](#) and ask if something was changed recently on the environment in question.

4. Test Data: Check that test data is valid. Live environments like Staging and Production rely on pre-existing data (QA users, access tokens).

5. New GitLab QA version: Check if a new GitLab QA version was released.

Failure examples can be seen in [Training Videos](#).

### Classify and triage the test failure

The aim of this step is to categorize the failure as either a stale test, a bug in the test, a bug in the application code, or a flaky test.

We use the following labels to capture the cause of the failure.

- ~"failure::investigating": Default label to apply at the start of investigation.
- ~"failure::stale-test": Stale test due to application change
- ~"failure::broken-test": Bug in the test
- ~"failure::flaky-test": Flaky test
- ~"failure::test-environment": Failure due to test environment
- ~"failure::bug": Bug in the application
- ~"failure::external-dependency": Failure due to an external dependency

Bugs blocking end-to-end test execution (due to the resulting quarantined tests) should additionally have severity and priority labels. For guidelines about which to choose, please see the [blocked tests](#) section of the [issue triage](#) page.

Note: It might take a while for a fix to propagate to all environments. Be aware that a new failure could be related to a recently-merged fix that hasn't made it to the relevant environment yet. Similarly, if a known failure occurs but the test should pass because a fix has been merged, verify that the fix has been deployed to the relevant environment before attempting to troubleshoot further.

### Stale test due to application change

The failure was caused by a change in the application code and the test needs to be updated.

- Include your findings in a note in the issue about the failure.
- Apply the ~"failure::stale-test" label.
- If possible, mention the merge request which caused the test to break, to keep the corresponding engineer informed.

See [Quarantining Tests](#)

## Bug in the test

The failure was caused by a bug in the test code itself, not in the application code.

- Include your findings in a note in the issue about the failure.
- Apply the ~"failure::broken-test" label.

See Quarantining Tests

## Bug in the application

The failure was caused by a bug in the application code.

- Apply ~"failure::bug" label to the test failure issue(s).
- Create a new issue link all related test failure issue(s) to this issue.
- Include your findings in a note in the bug issue about the failure.
- Add the steps to reproduce the bug and expected/actual behavior.
- Apply the ~"type::bug" label, and cc-ing the corresponding Engineering Managers (EM), QEM, and SET.
- Apply the ~"found by e2e test" label to the bug issue (or bug fix MR if it is fixed immediately) to indicate the bug was found by the end-to-end test execution.
- If the problem adheres to the definition of a transient bug, apply the ~"bug::transient" label as well.
- If there is an issue open already for the bug, use this issue instead and apply the above steps.
- Communicate the issue in the corresponding Slack channels.
- Quarantine the test right after the bug issue has been created. Leave a note on the bug issue with the link to the quarantined test and mention that it should be un-quarantined with the fix.
- When the reason for quarantining a test is because of a low severity bug in the code which will not be fixed in the upcoming couple of releases, add the ~"quarantine" , quarantine with type and ~"failure::bug" labels to the test failure issue(s).
- When the bug is fixed, the related quarantined tests should be unquarantined and verified as well. The bug issue and all related test failure issues should be closed out together.

Note: GitLab maintains a daily deployment cadence so a breaking change in master reaches Canary and Production fast. Please communicate broadly to ensure that the corresponding Product Group is aware of the regression and action is required. If the bug is qualified for dev escalation (example: priority::1/severity::1 issue that blocks the deployment process), consider involving On-call Engineers in the dev-escalation channel. To find out who's on-call follow the links in the channel subject line.

To find the appropriate team member to cc, please refer to the Organizational Chart.

See Quarantining Tests

## Flaky Test

Read more:



- What is a flaky test?
- What are the potential causes for a test to be flaky?

## Process

- Include your findings in a note in the failure issue.
- Apply the ~"failure::flaky-test" label to the failure issue.
- Apply a ~"flaky-test::" scoped label to the failure issue.

Flakiness can be caused by a myriad of problems. Examples of underlying problems that have caused us flakiness include:

- Not waiting appropriately for pages to load or transitions from one state to another to complete.
- Animations preventing tests from interacting with elements.
- Non-independent tests (i.e. test A passes when run first, but fails otherwise).
- Actions not completing successfully (e.g. logging out).

For more details, see the list with example issues in our [unhealthy tests documentation](#).

## See Quarantining Tests

### Failure due to test environment

The failure is due external factors outside the scope of the test but within a test environment under GitLab's control. This could be due to environments, deployment hang-ups, or upstream dependencies within GitLab's control.

- Include your findings in a note in the issue about the failure.
- Apply the ~"failure::test-environment" label.
- Identify the general category of improvement and add the failure issue to the appropriate Test Reliability issue listed within the [Improve test environment reliability and reduce flaky/transient test failures tracking issue](#).

A job may fail due to infrastructure or orchestration issues that are not related to any specific test. In some cases these issues will fail a job before tests are ever executed. Some examples of non-test related failures include:

- Failed to download container from GitLab Container Registry
- Failed to complete orchestration of a Geo cluster
- CI runner timeouts
- 500 error while uploading job artifacts
- Expired tokens (For more information, check out [how to Rotate Credentials](#))

### Failure due to external dependency

The failure is due to an external dependency the test is reliant on, but is outside of GitLab's control. This could be due to outages in external package management systems, or outages in third-party integrations. If possible, external dependencies should be avoided to increase the

reliability of the test suite.

- Include your findings in a note in the issue about the failure.
- Apply the ~"failure::external-dependency" label.
- Include a link to the external dependency's outage notice, if available.

Some examples of external dependency failures could include:

- Outages in package or container management systems such as registry.npmjs.org, RubyGems.org, NuGet or dockerhub
- Outages in third-party integrations such as Zuora

Notify relevant groups about the failure

Failure needs escalation

If the failure is in a smoke test, it will block deployments. Please inform the release managers of the root cause and if a fix is in progress by Quality. On GitLab.com, you can use @gitlab-org/release/managers. In Slack, you can use @release-managers.

If the failure could affect the performance of GitLab.com production, or make it unavailable to a specific group of users, you can declare an incident with /incident declare in the production Slack channel, this will automatically prevent deployments (if the incident is at least an S2).

Notify group in all cases

Please also raise awareness by looping in the appropriate team members from the product group, such as SET or EM. The SET/EM can be identified by looking for who is assigned to that stage/group. Many tests are tagged with a productgroup which will help with identification. You may also want to post in Quality's Slack channel quality depending on the impact of the failure.

Following up on test failures

Fixing the test

If you've found that the test is the cause of the failure (either because the application code was changed or there's a bug in the test itself), it will need to be fixed. This might be done by another SET or by yourself. However, it should be fixed as soon as possible. In any case, the steps to follow are as follows:

- Create a merge request (MR) with the fix for the test failure.
- Apply the ~"Pick into auto-deploy", ~"priority::1", and ~"severity::1" labels if the fix is urgent and required to unblock deployments.

If the test was flaky:

- Confirm that the test is stable by ensuring it passes 3 to 5 times while in quarantine.

> Note The number of passes needed to be sure a test is stable is just a suggestion.

> You can use your judgement to pick a different threshold.

If the test was in quarantine, remove it from quarantine.

## Quarantining Tests

The aim of quarantining a test is not to get back a green pipeline, but rather to reduce the noise (due to constantly failing tests, flaky tests, and so on) so that new failures are not missed. If you're unsure about quarantining a test ask for help in the quality Slack channel, and then consider adding to the list of examples below to help future pipeline triage DRIs.

Examples of when to quarantine a test:

- Bug in test code (~"failure::broken-test"), and a fix won't be ready for review within 24 hours
- Stale test (~"failure::stale-test"), and a fix won't be ready for review within 24 hours

Examples of when not to quarantine a test:

- Test failure is environment-related (~"failure::test-environment"), and neither the application code nor test code are the cause of the failure
- A merge request to fix the failure will be ready for review within 24 hours
- A flaky smoke test.
  - The :smoke tag should be removed from the test to prevent it running with the smoke suite, but still allowed to run elsewhere while the flakiness is under investigation or being worked on to unblock deployment.
  - Create an issue to follow up and restore the :smoke tag as soon as possible. Tests at the :smoke level should be given priority when addressing flakiness within our test suites.
  - Create a TODO note in the test as a reminder with a link to the previously created issue url. For example:
    - TODO restore :smoke tag and close <https://gitlab.com/gitlab-org/gitlab/-/issues/>

> Note The time limit for the fix is just a suggestion.

> You can use your judgement to pick a different threshold.

## Fast quarantine

If a test is failing and you have good reason to quarantine it quickly you can fast quarantine it. Fast quarantine can be done by a single person, without requiring review by another engineer.

You should use fast quarantine to unblock deployment pipelines and MRs if the failure is disruptive and you've ruled out bugs as the cause of the failure (e.g., you've identified that the test is stale, or flaky).

> Note: Failing gdk-instance job in the e2e: test-on-gdk child pipeline will block MR pipelines as well as scheduled master pipelines. Failures of those tests are good candidates for fast quarantine.

After fast quarantining, please follow the long-term quarantine process below.

## Long-term quarantine

- Add the :quarantine metadata to the test with a link to the issue (see quarantined test types)

> Note If the example has a before hook, the quarantine metadata should be assigned to the outer context to avoid running the before hook.

- Create a merge request using the Quarantine end-to-end Test template.
  - The merge request shall have the labels: ~"Quality", ~"QA", ~"type::bug".
  - The merge request may have auto-deploy labels: ~"Pick into auto-deploy", ~"priority::1", and ~"severity::1". Please note that this is reserved for emergency cases only, such as blocked deployments, as it will delay all other deployments by around two hours.
  - The merge request may have stage or group labels. E.g. ~"devops::create" ~"group::source code".
  - The merge request shall have the current milestone.
  - The merge request shall link to (but not close) the failure issue.
- The failure issue should remain open and be assigned a DRI for fixing, re-evaluating, or dequarantining the test as needed.

To be sure that the test is quarantined quickly, ask in the quality Slack channel for someone to review and merge the merge request, rather than assigning it directly.

Here is an example quarantine merge request.

> Note Be aware that for blocked release cycle auto deploy with quarantined test will not work. During canary deployment we trigger tests against both staging and canary. Spec is not quarantined on staging yet because quarantined MR didn't reach it. That leads to environment circular dependency problem.

>

> In that case the Release Managers should decide whether it is possible to rollback Staging, or whether to manually deploy the quarantined MR to Staging. Manually deploying the fix to Staging will reduce the amount of mixed-version testing but may be the only option to allow deployment pipelines to continue.

## Quarantined Test Types

If a test is placed under quarantine, it is important to specify why. By specifying a quarantine type we can see quickly the reason for the quarantine.

Below is a list of quarantine types you may use, and their purpose. If the reason for quarantine is not listed below, please consider contributing to this list:

| Quarantine Type | Description                                                                          |
|-----------------|--------------------------------------------------------------------------------------|
| :flaky          | This test fails intermittently                                                       |
| :bug            | This test is failing due to a bug in the application                                 |
| :stale          | This test is outdated due to a feature change in the application and must be updated |

to fit the new changes |  
 | :broken | This test is failing because of a change to the test code or framework |  
 | :waitingon | This test is quarantined temporarily due to an issue or MR that is a prerequisite for this test to pass |  
 | :investigating | This test is a :flaky test but it might be blocking other MRs and so should be quarantined while it's under investigation |  
 | :testenvironment | This test is failing due to problems with the test environment and will not be fixed within 24 hours |  
 | :dependency | This test is failing due to an external dependency the test is reliant on, but is outside of GitLab's control |

> Note: Be sure to attach an issue to the quarantine metadata. We use this issue for tracking the average age of the quarantined tests.

## Examples

```
ruby
it 'is flaky', quarantine: { issue: 'https://gitlab.com/gitlab-org/gitlab/issues/12345', type: :flaky }
it 'is due to a bug', quarantine: {
  issue: 'https://gitlab.com/gitlab-org/gitlab/issues/12345',
  type: :bug
}
context 'when these tests rely on another MR', quarantine: {
  type: :waitingon,
  issue: 'https://gitlab.com/gitlab-org/gitlab/mergerequests/12345'
}
```

## Nested contexts

You should apply the quarantine tag to the outermost describe/context block that has tags relevant to the test being quarantined.

```
ruby
Good
RSpec.describe 'Plan', :smoke, quarantine: { issue: 'https://gitlab.com/gitlab-org/gitlab/issues/12345', type: :flaky } do
  describe 'Feature' do
    before(:context) do
      This before(:context) block will only be executed in smoke quarantine jobs
    end
  end
end
```

```
Bad
RSpec.describe 'Plan', :smoke do
  describe 'Feature', quarantine: { issue: 'https://gitlab.com/gitlab-org/gitlab/issues/12345',
```

```
type: :flaky } do
  before(:context) do
    This before(:context) block could be mistakenly executed in quarantine jobs that don't
    have the smoke tag
  end
end
end
```

## Dequarantining Tests

Failing to dequarantine tests periodically reduces the effectiveness of the test suite. Hence, the tests should be dequarantined on or before the due-date mentioned in the corresponding issue.

Before dequarantining a test:

- If the test failure was originally discovered in gitlab-org/gitlab nightly schedule pipeline, MR or master pipeline, please make sure that the test passes on your local against GDK with latest code and that it has passed a few times in the nightly pipeline's quarantine job for that test.
- If the test failure was originally discovered in staging, canary or production pipeline, please make sure that the test passes in the CI pipeline against that environment. You can trigger a CI pipeline against a live environment by clicking "Run Pipeline" button on the staging, canary or production pipelines page and setting the RELEASE variable to the release that has your changes. See Running GitLab-QA pipeline against a specific GitLab release for instruction on finding your release version created and tagged by the Omnibus pipeline.

To dequarantine a test:

- Create a merge request that removes the :quarantine tag using the Quarantine End to End Test MR template.
- Close the issue created as part of the quarantining process.

As with quarantining a test, you can ask in the quality Slack channel for someone to review and merge the merge request, rather than assigning it.

## Re-evaluating tests

If the due date of a failing test issue is reached, you should re-evaluate if the failing test should really be covered at the end-to-end test level, or if it should be covered in a lower level of the testing levels pyramid.

If you decide to delete the test, open a merge request to delete it and close the test failure issue. In the MR description or comment, mention the stable counterpart SET for the test's stage for their awareness. Then open a new issue to cover the test scenario in a different test level.

If you decide the test is still valuable but don't want to leave it quarantined, you could

replace :quarantine with :skip, which will skip the test entirely (i.e., it won't run even in jobs for quarantined tests). That can be useful when you know the test will continue to fail for some time (e.g., at least the next milestone or two).

### Quarantined test report

Every week, a list of the current long-term quarantined E2E tests in both the GitLab and CustomersDot project is reported to the quality-reports Slack channel.

This report is generated from the Test Platform Department's toolbox project.

### Training videos

These videos walking through the triage process were recorded and uploaded to the GitLab Unfiltered YouTube channel.

- Quality Team: Failure Triage Training - Part 1
  - Covers the basics of investigating pipeline failures locally.
- Quality Team: Failure Triage Training - Part 2
  - Continued discussion with a focus on using Docker containers that were used in the pipeline that failed.
- Quality Engineering On-call Rotation and Debugging QA failures (private video on GitLab Unfiltered)
  - Overview of QE on-call rotation process, GitLab deployment process and how to debug failed E2E specs with examples.
- Quality Engineering: Test environments show and tell
  - A show and tell presentation that gives an overview of our test environments. Originally presented for an audience of our counterparts.
- Runner Taskscaler and Fleeting Test Plan Discussion
  - A high level discussion on the new runner architecture, including the new taskscaler and fleeting, the components replacing docker-machine for runner autoscaling.

---

## SECTION: engineering/testing/risk-mapping.md

The Quality Engineering Sub-Department helps facilitate the risk mapping process. This requires the participation of Product Management, Development, UX, and the Quality team to develop a strategic approach to risk and mitigation planning.

### Goals

Utilise the Risk Map as a tool to:

- Understand the risks the counterpart team faces
- Increase transparency on mitigation plans
- Effectively allocate limited resources
- Collaborate strategically in improving Quality

### Risk Management

Risk Management is the process of identifying risks and their impact to all areas of the business and the organization. Identifying these risks could greatly assist teams in making and carrying out decisions that will minimize their adverse effects. By focusing attention on the risks and committing the necessary resources to control and mitigate them, a team will better protect itself from uncertainty as well as increase product stability, quality and performance.

The Risk Management cycle consists of six phases:

- Awareness
- Identification
- Evaluation
- Control
- Implementation
- Monitoring

How to build a Risk Map

General Risk Map

To raise awareness and start identifying risks, first create two lists - Areas and Facets.

Example:

| Area |  
| ----- |  
| Team |  
| Product |  
| Infrastructure |  
| UX |  
| Quality |  
| Community |  
| Customers |  
| ... |

| Facet |  
| ----- |  
| Expertise |  
| Performance |  
| Test Coverage |  
| Migrations |  
| Scalability |  
| Stability |  
| Experience |  
| Data |  
| ... |

Combine Areas and Facets to produce a table where each cell is an associated risk. Example:

|        |       |         |                |       |         |       |
|--------|-------|---------|----------------|-------|---------|-------|
| Areas  | Team  | Product | Infrastructure | UX    | Quality | ...   |
| -----  | ----- | -----   | -----          | ----- | -----   | ----- |
| Facets | ----- | -----   | -----          | ----- | -----   | ----- |



| Expertise | Concentrated domain knowledge | | | Insufficient SETs | |  
 | Performance | Burn out | Often miss SLO/SLAs | Outages | | Flaky tests | |  
 | Test Coverage | | | Test environment is hard to reproduce | | |  
 | Migrations | | Customers migrate with difficulties from previous product/platform | Database migrations often causes outages | | |  
 | Scalability | | | | Downgrade of Quality efforts to meet demands | |

After naming the risks by integrating different Areas and Facets to help brainstorm, the next phase - Evaluation - requires understanding which impact it may cause and estimations on each risk's impact level and probability of occurrence. The product of these two dimensions will determine the risk's score (the higher the score the higher the priority).

### General Risk Map, an Example

#### Map key

- Impact - what happens if the risk is not mitigated or eliminated
- Impact level - Rate 1 (LOW) to 5 (HIGH)
- Probability - Rate 1 (LOW) to 5 (HIGH)
- Priority - Impact x Probability. Address highest score first.
- Mitigation - what could be done to lower the impact or probability

| Risk Area                                          | Risk Description                                                                       | Impact                                   |
|----------------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------|
| Impact level   Probability   Priority   Mitigation |                                                                                        |                                          |
| -----   -----   -----   -----                      |                                                                                        | -----                                    |
| -----   -----   -----   -----                      |                                                                                        |                                          |
| Team/Stability                                     | Burn out                                                                               | Low                                      |
| productivity and attrition                         | 5   2   10                                                                             |                                          |
| Team/Scaling                                       | Inefficient team member onboarding                                                     |                                          |
| Prolonged low productivity                         | 3   2   6                                                                              |                                          |
| Team/Expertise                                     | Concentration of knowledge                                                             | Missed                                   |
| SLO/SLA                                            | 4   3   12                                                                             |                                          |
| Customer                                           | Broken promises                                                                        | Reduced                                  |
| GMAU                                               | 5   2   10                                                                             |                                          |
| Customer                                           | Eroded trust with the community                                                        | Fewer                                    |
| community contributions                            | 5   1   5                                                                              |                                          |
| Product/Scope                                      | Not enough knowledge about how the product is being used                               | Reduced                                  |
| [METRIC]                                           | 3   3   9                                                                              |                                          |
| Product/Scope                                      | Increase of security vulnerabilities due to having many different areas of the product | Loss of confidence / revenue   5   1   5 |
| Product/Data                                       | User metrics and activity metrics are incomplete and hard to track                     | Inaccurate sensing data   4   3   12     |
| Quality                                            | Downgrade quality to meet maturity targets                                             | Escaped                                  |
| bugs                                               | 5   3   15                                                                             |                                          |
| Quality                                            | Uncertain test coverage                                                                |                                          |
| Difficult to prioritise test effort                | 3   3   9                                                                              |                                          |
| Feature/Performance                                | Low performance due to                                                                 | Low customer satisfaction, reduced       |
| [METRIC]                                           | 5   4   20                                                                             |                                          |
| Feature/Testability                                | Hard to drive real world test scenarios                                                | Escaped                                  |
| bugs                                               | 4   4   16                                                                             |                                          |

| Feature/Dependencies | Cross-group work not being prioritised in a timely manner | Delayed deliverables, reduced customer satisfaction, reduced team productivity | 3 | 3 | 9 | |

Teams can iterate on this exercise by expanding it to their Product Categories or even to the Feature level, having a more granular understanding of the risks.

## Risk Control

After evaluating the risk impact and probability, the Control and Implementation phases require to create mitigations for each risk in order to reduce the negative effects of its impact. Mitigations are strategies for planning work around the impact area. Some strategic ways to deal with risk are:

- Risk avoidance - used when the consequences are deemed too high to justify the cost of mitigating the problem.
- Risk acceptance - accepting a risk for a given period of time to prioritize mitigation efforts.
- Risk transfer - allocates risks between different teams, consistent with their capacity to protect against or mitigate the risk.
- Risk monitoring - keep track of projects and the associated risks for changes in the impact of the associated risks.

Tracking and monitoring risks and the work being done towards their mitigation is up to the team preferred workflow.

## Risk Mapping Tool

The Risk Mapping Tool helps teams easily visualize risks and planned mitigations. Teams may implement this if desired to avoid having to manually create a risk map. It supports the risk mapping process which enables teams to be more transparent, collaborative and efficient when it comes to strategically improve overall quality in a productive way.

Setting up the Risk Mapping Tool is not a requirement, but may be helpful for quick visualizations of risk status. If metrics are in place to measure risk status, the Risk Mapping Tool can more easily expose these.

The Risk Mapping Tool belongs to the Projects maintained by Quality and could be a part of the Quad-Planning process feeding into the Test Governance practices by facilitating the test planning process with an initial risk analysis.

To install the Risk Mapping tool, please follow the README instructions.

If desired, a team or group could also manually input these in a visual risk map. Here's an example of a complete visual risk map.

---

## SECTION: engineering/testing/test-coverage.md

Offline environments / Airgapped GitLab QA scenario

The Test Platform Department has a GitLab QA scenario that supports offline environment / air-gapped testing.

The scenario Test::Instance::Airgapped is part of GitLab QA test scenarios. The suite runs against a test environment including Gitaly Cluster which have been configured using iptables to drop traffic other than specific ports which allow our test access to the test instances.

## Test run schedule

This is triggered on the gitlab-org/gitlab nightly schedule pipeline, where a near full suite of tests for CE and EE are executed (exclusions are made for tests of product features which depend on network connectivity such as importing data from external sources). Results of the gitlab-org/gitlab nightly schedule pipeline can be found in the Allure report, where test states such as failures can be filtered on. Nightly pipelines are visible at the gitlab-org/gitlab nightly schedule pipelines page (internal only). The offline environment / airgapped test job names are ce:airgapped and ee:airgapped.

## Other reference guides

Secure stage has additional testing to test that analyzers can execute in an offline fashion. More information on secure tests(internal only).

Otherwise for setting up an offline environment for testing, the Getting started with an offline GitLab Installation guide can be followed. Instructions for working with secure scanners can be found in the Offline environments guide.

## GitLab Upgrades

The goal of GitLab Upgrades test coverage is to ensure that a customer following the upgrade path will be successful.

To achieve the best coverage, Test Platform follows the Test Pyramid approach by shifting left to unit tests without build environments in merge requests and going up to system level testing with actual environments being built:

- 1. Lower level testing - Multi-version migration upgrade jobs
- 1. System level testing - Single-node/Docker upgrades
- 1. System level testing - Multi-node/Self-Managed upgrades

### Multi-version migration upgrade jobs

| Upgrade path scenarios                                            | Example |
|-------------------------------------------------------------------|---------|
| -----                                                             |         |
| Latest update stop · GitLab Merge Requests   16.7.7 · MR in 16.11 |         |

db:migrate:multi-version-upgrade  
validates that the migrations pass for multi-version upgrade from the latest required upgrade

stop to the author's working branch.

It allows catching migration error(s) at unit-level without building an environment.

Test job runs Database migrations against PostgreSQL dump created from the latest known GitLab version stop with test data.

#### GitLab QA update scenario

| Upgrade Path                                                                              | Description                          | Upgrade Path                           | Job Name                                        | Example Job Link |
|-------------------------------------------------------------------------------------------|--------------------------------------|----------------------------------------|-------------------------------------------------|------------------|
| From latest update stop in previous major version to current package built from branch/MR | Major-1 · update stops · pre-package | update-major                           | 15.11.13 · 16.1.6 · 16.3.7 · 16.7.7 · 16.11-pre |                  |
| From latest minor release to current package built from branch/MR                         | Minor-1 · pre-package                | update-minor                           | 16.10.1 · 16.11-pre                             |                  |
| From current patch version to current package built from branch/MR                        | Patch-1 · pre-package                | update-patch                           | 17.7.6 · 17.7.7-pre                             |                  |
| From current package built from branch/MR package to next stable GitLab release           | pre-package · N+1                    | update-from-patch-to-stable            | 17.7.7-pre · 17.8.4                             |                  |
| From current internal patch version to current package built from branch/MR               | Internal Patch-1 · pre-package       | update-patch-from-internal-to-internal | 17.7.7-internal0-0 · 17.7.7-pre                 |                  |

GitLab QA has a `Test::Omnibus::UpdateFromPrevious` scenario that verifies update from the previous (major|minor|patch) version to the current GitLab version (scenario code).

Additionally, a `Test::Omnibus::UpdateToNext` GitLab QA scenario supports testing the upgrade path from a development version to the next available stable release.

#### Test run schedule

1. `Test::Omnibus::UpdateFromPrevious` scenario is run with:

- `e2e:test-on-omnibus-ee` / `e2e:test-on-omnibus-ce` jobs which executes from a scheduled pipeline every 2 hours against GitLab master.
- `e2e:test-on-omnibus-nightly` job which executes from a nightly scheduled pipeline against GitLab master.
- Backport merge requests to test upgrades from stable releases to pre-release packages built within the MR.

1. `Test::Omnibus::UpdateToNext` scenario is run with:

- Backport merge requests to test upgrades from pre-release packages built within the MR to stable releases.

#### Performance environments nightly upgrades

Framework and Performance Enablement teams support test performance environments listed on Reference Architecture page.

These environments are built with GitLab Environment Toolkit and are upgraded daily or weekly depending on environment to the latest nightly image.

Detailed process is described on Performance and Scalability page.

## Upgrade Tester

| Upgrade path scenarios                 | Example                                                           |
|----------------------------------------|-------------------------------------------------------------------|
| ----- -----                            |                                                                   |
| -----                                  |                                                                   |
| Latest update stop · GitLab Nightly    | 16.7.7 · nightly                                                  |
|                                        |                                                                   |
| Latest GitLab release · GitLab Nightly | 16.10.1 · nightly                                                 |
| Custom path scenarios                  | 15.0.0, 15.0.5, 15.4.6, 15.11.13, 16.1.6, 16.3.7, 16.7.7, 16.10.0 |

Focused on building and testing different upgrade paths using the Reference Architectures, the Upgrade Tester pipelines build and upgrade environments starting at a specified version and ending at either the latest nightly package or a specific version. For each upgrade the path used to upgrade differs depending on the start and end versions used. For example, when starting with version 16.0.0 the upgrade path would be 16.0.0, 16.1.6, 16.3.7, 16.7.7, nightly.

More information can be found within the Upgrade Tester project about the schedule and Reference Architecture types that are used for testing. Test results are reported to qa-upgrade-results channel in Slack and monitored by Self-Managed Platform team.

## Work in progress

Test Platform team is working on improving GitLab upgrades coverage and this effort is tracked in epic12458.

---

## SECTION: engineering/training/\_index.md

## Resources

### Secure coding best practices

It is important that all developers are aware of secure coding best practices and refresh this knowledge periodically. This is tracked via Secure Coding Training Guidelines.

### Ruby on Rails Performance Training

The materials from an earlier Ruby on Rails performance workshop can be found on internally shared Google drive.

## Video Sessions

| Day | Topics | Video Links |
|-----|--------|-------------|
| --- | -----  | -----       |

| Session - Day 1 | Intro and overview | Monday Wednesday |  
| Session - Day 2 | Tools | Monday Wednesday |  
| Session - Day 3 | SQL and N+1 Troubleshooting | Monday Wednesday |  
| Session - Day 4 | Queueing Theory | Monday Wednesday |

## Database

Here is the information of a PostgreSQL query optimization bot at GitLab - Joe: Blueprint and Design.

## Frontend

### Frontend Masters

Frontend Masters allows you to advance your skills with in-depth, modern frontend engineering courses.

Team members can use the Growth and Development Fund to request a subscription by following the process outlined at <https://about.gitlab.com/handbook/total-rewards/benefits/general-and-entity-benefits/growth-and-development/>.

## GraphQL

We use GraphQL alongside our REST API at GitLab and are increasingly adding new features to the GraphQL API over time.

The GraphQL API can be added to by anyone, including community members. We have a group of self-selected team members who are willing to help with any GraphQL questions you may have. You can get in touch with them by mentioning @gitlab-org/graphql-experts in any GitLab issue or merge request.

You can read more information about GraphQL at GitLab here:

- Getting started with GitLab GraphQL API
- GraphQL frontend guide
- GraphQL API style guide
- Introduction to GraphQL ([graphql.org](https://graphql.org))

## Engineering Management

One of the paths for engineers to progress in their career is a Management path. To explain more about the role of Engineering Managers in GitLab, there is a series of conversations with team members explaining details of their work and their experience transitioning to management [here](#).

## IC Leadership

Staff+ engineers and ICs aspiring to become Staff can benefit from resources and recommendati